zRA 逐页演讲稿

20页详解版 · 建议汇报时间20分钟

本稿与PPT页码一一对应；每页讲稿也已写入PPT备注。时间分配为建议值，实际时长随语速和图表停顿调整。

复现数据来自原PPT截图，本次未重新运行实验；技术数据依据上传论文核对。

页	主题	用时	累计
1	论文题目与汇报人	25秒	00:25
2	汇报路线：从验证权限到公开证据	25秒	00:50
3	远程证明验证什么：模型与信任边界	55秒	01:45
4	场景驱动：让第三方独立核验设备	45秒	02:30
5	研究缺口：公开结果与独立核验的区别	55秒	03:25
6	从交互到独立：两种消息流	55秒	04:20
7	设备侧信任边界：测量与证明分工	60秒	05:20
8	三大组件：各自承担什么任务	55秒	06:15
9	四阶段：离线准备与在线证明分开	70秒	07:25
10	Setup：响应承诺与两层 Merkle 树	90秒	08:55
11	Attestation：公开输入、见证与约束	95秒	10:30
12	Verification：四项检查缺一不可	75秒	11:45
13	安全分析：保护机制与覆盖边界	60秒	12:45
14	实验设置与存储：规模增长体现在哪	70秒	13:55
15	原型链路：从电路到 Sepolia 合约	55秒	14:50
16	设备端耗时：论文结果与已有复现	100秒	16:30
17	可扩展性：制造商轻了，全网仍有成本	70秒	17:40
18	能耗与设备负担：快不一定更省电	65秒	18:45
19	论文评价：贡献、代价与后续方向	65秒	19:50
20	谢谢聆听与问答	10秒	20:00

第01页 论文题目与汇报人

建议用时：25秒

大家好，我们今天汇报的是 NDSS 2024 论文《From Interaction to Independence》，中文可以理解为“从交互走向独立：利用 zkSNARK 实现透明、非交互的远程证明”。

这篇论文提出 zRA，关注的核心问题是：设备的软件完整性，能不能由任何第三方直接核验，而不必依赖掌握设备秘密的特定验证者？

接下来我们围绕协议设计、原型实验和安全边界展开介绍。

内容出处：论文首页；原PPT第1页

第02页 汇报路线：从验证权限到公开证据

建议用时：25秒

整个汇报分成四部分：首先说明传统远程证明为什么依赖特定验证者，然后用八分多钟讲清 zRA 怎样连接可信测量、响应承诺和零知识证明，接着分析设备端的计算、存储与能耗，最后讨论它的贡献与局限。

听完以后，希望大家能够回答两个问题：zRA 到底消除了哪一部分交互，以及这些交互消失后，系统仍然需要信任什么。

内容出处：原PPT目录；论文I – IX

第03页 远程证明验证什么：模型与信任边界

建议用时：55秒

远程证明可以理解为：验证者向一台远端设备询问，它当前的软件测量是否符合预期，而设备需要给出能够核验的证据。

传统流程通常先发送一个新挑战，再由设备把挑战和自身测量组合成响应，验证者使用预先掌握的密钥、内部状态或参考测量检查响应，这样新挑战就能把一次测量和本次证明联系起来。

这里有一个容易忽略的前提：即使攻击者控制设备的普通软件，也不能控制产生真实测量的信任锚，否则后面的密码学检查只是在验证一个不真实的输入。

论文使用的最小信任锚是 ROM 和 MPU，分别用于保护测量代码与内部受保护状态，因此证明结论应限定为所测状态和所用挑战下的完整性，而不是设备未来始终安全。

内容出处：论文I、II、附录B/C；PDF第1、3、15 – 16页

第04页 场景驱动：让第三方独立核验设备

建议用时：45秒

以智能门锁为例，制造商之外的云服务、合作方和普通用户，都可能想知道它运行的软件是否被修改，但他们通常没有设备的内部秘密，也不适合各自向制造商申请一次人工或在线核验。

zRA 的思路是，制造商提前准备认可响应的承诺树，之后周期性公布共享挑战；设备自己读取挑战、完成可信测量并提交证明，而第三方只读取公开信息完成检查。

需要注意，设备仍然访问公开节点并提交消息，“ 非交互 ”说的是不再和掌握特权信息的可信方逐次往返。

这样，多个第三方就可以围绕同一份公开证据独立得出结果，验证权限从特定服务扩展到了公开参与者。

内容出处：论文III、IV-A；原PPT第4页场景

第05页 研究缺口：公开结果与独立核验的区别

建议用时：55秒

已有方案已经尝试用区块链、代理验证者或异步机制改善远程证明，所以这篇论文并不是第一次把证明结果放到链上。

比如 SCRAPS 通过代理验证者和私有链支持异步证明，但公众仍然依赖代理的验证结果；PROVE 支持公开验证，却依赖可信事件、日志和链式历史，论文将它与独立验证当前证明区分开来。

可以把这一差别理解为：看到“ 某个机构说设备通过了 ”，和拿到“ 我自己能检查设备为何通过 ”的证据，是两种不同能力。

zRA 的目标是让验证者不需要设备的预共享密钥或参考测量，直接验证当前证明，但它仍然依赖离线准备的认可响应以及设备侧的信任锚。

表格反映的是原论文对这些代表性方案的比较，不能据此说所有传统 RA 都具有完全相同的实现限制。

内容出处：论文Table II、III、VIII；PDF第3 – 4、13页

第06页 从交互到独立：两种消息流

建议用时：55秒

左边的传统流程中，验证者为某台设备发送挑战，设备返回状态相关响应，然后由具备特权知识的验证者计算或查询预期响应来比较，这个验证者因此处在每次证明的关键路径上。

右边的 zRA 把这条路径拆开：制造商只周期性公布一个全局挑战，设备自主生成证明并提交，任何人或者智能合约都能使用公开参数检查。

制造商不需要针对每台设备回答“ 它现在应该返回什么 ”，第三方也不需要拿到这个正确响应本身。

这里的透明指检查过程和依据可公开复核；零知识指证明不展示响应及成员路径，而不是隐藏设备公钥和挑战，因为这些值恰好必须公开绑定。

接下来要解决的问题是，这个不公开的正确响应究竟从哪里来，以及如何防止被篡改的软件伪造它。

内容出处：论文I、IV-A、Fig.2；PDF第1、5页

第07页 设备侧信任边界：测量与证明分工

建议用时：60秒

这个问题由设备的 ROM 和 MPU 来回答：ROM 中的固定程序取得测量，MPU 使攻击者控制的普通软件不能直接读取受保护的内部状态。

附录给出一个例子，把内部状态、挑战和测量校验值一起哈希，得到响应，所以如果软件测量发生变化，输出就不会是制造商事先认可的那个响应。

响应交给普通软件层以后，普通软件执行的是承诺与 Merkle 路径检查，随后生成证明并提交；论文不要求这部分 ZK 计算运行在额外的可信执行环境中。

要特别区分两层保证：信任锚保证响应确实来自可信测量，而证明系统保证交给它的响应符合已经认可的承诺，电路本身没有重新测量整台设备。

response 在电路中作为私有见证，并不意味着论文要求对它配置独立安全存储，论文认为它的哈希形式不会直接暴露内部状态。

内容出处：论文Fig.4、V-A、附录C；PDF第7 – 8、16页

第08页 三大组件：各自承担什么任务

建议用时：55秒

zRA 把三种工具组合起来，但它们处理的是不同问题：承诺把正确响应和设备身份、挑战绑定起来，Merkle 树把大量承诺组织到一个公开根，zkSNARK 则证明“ 我知道一个合格见证 ”，同时不把见证展示给验证者。

Merkle 路径只需逐层提供兄弟节点和左右位置，不需要重新读取并哈希整棵树，因此树越大，单次检查主要随着树高增长。

这里采用 Poseidon，是因为它适合放进算术电路，而叶子里保存的是身份、挑战和响应的哈希承诺，不是直接公开设备的敏感内存。

对证明系统，完备性表示正确见证能够通过，可靠性阻止没有合格见证的攻击者伪造通过，零知识则使检查者无需学习见证内容。

这三层结合起来，才把“ 只有掌握设备秘密的人能比较响应 ”转换成“ 任何人都能检查公开证明 ”。

内容出处：论文II、IV-A、Listing 1/2；PDF第3、5 – 6、9页

第09页 四阶段：离线准备与在线证明分开

建议用时：70秒

第一阶段由制造商离线完成，它选定未来挑战，计算每台设备在认可状态下的正确响应，构建承诺树，并公开后续验证所需的根、树和证明参数。

第二阶段是挑战更新：制造商不会一次公开整个未来序列，而是根据证明周期只发布一个新挑战，同一周期内所有设备使用这个挑战。

第三阶段由设备执行，设备读取最新挑战，通过信任锚得到响应，再结合成员路径生成证明，并把证明和公开输入签名后提交。

第四阶段是公众或合约验证，除了检查 zkSNARK，还要确认提交身份、根是否获认可，以及挑战是否最新，不能把这些外围条件省略成一个 verifyProof 调用。

虽然每次证明不需要制造商处理设备请求，但制造商或者后面介绍的阈值参与方仍然需要按时发布挑战，所以离线准备、周期性更新和逐设备在线服务应分别讨论。

内容出处：论文IV-A、Fig.2、Alg.1 – 3；PDF第5 – 7页

第10页 Setup：响应承诺与两层 Merkle 树

建议用时：90秒

这一页把 Setup 展开：对第 k 台设备、第 i 个挑战，制造商先计算认可响应，然后把设备公钥、响应和挑战一起输入 Poseidon，得到一个承诺叶子，这使某个响应不能随意换到另一个设备或另一个挑战上使用。

下层树对应一台设备的 m 次未来证明，因此每个设备有自己的根；上层树把 n 个设备根再作为叶子，最终得到所有设备共用的全局根 R，单次证明的路径会穿过这两层结构。

总路径高度是向上取整的 log_2m 加上向上取整的 log_2n，所以树高四十可以容纳大约一百万台设备、每台一百万次证明，前提是按两层树的实际填充和高度安排容量。

公开的是承诺树及其根，未发布的未来挑战仍须保密，因此原 PPT 中“ 只有根公开 ”需要改成更准确的表述。

还要看到，这种设计把制造商的反复在线计算转移到了离线预计算，全部响应和叶子的规模仍然随 n 乘 m 增长；这一页使用论文协议公式的输入顺序，实际 Listing 2 使用公钥、挑战、响应的顺序，工程实现必须确保建树与电路一致。

内容出处：论文IV-A1、Fig.3、Alg.1；PDF第5 – 6页

第11页 Attestation：公开输入、见证与约束

建议用时：95秒

这是协议最关键的一页，先区分哪些值公开：根据原型电路的 public 列表，只有 root、pubAddr 和 challenge，也就是全局根、设备身份和当前挑战，所以 response 不应出现在公开输入里。

私有见证包含响应、Merkle 兄弟节点以及左右方向位；电路先按 Listing 2 的顺序计算 Poseidon，再根据方向位决定每层哈希的左右顺序，最后断言计算出的根与公开根相同。

证明通过后，验证者知道设备拥有一个响应，使它对应的承诺确实属于认可树，但并不知道这个响应与路径的具体数值。

这里没有把整个内部状态直接作为电路的原始输入，更不能说电路独立证明设备正在运行每一条正确指令，真实测量的保证来自前一页介绍的信任锚。

符号也要分开，设备公钥写成 pk_k，而证明系统的证明密钥写成 d_p，证明密钥及验证密钥可以公开，这样就不会像原稿一样用同一个 pk 同时表示两种完全不同的对象。

内容出处：论文式(1)、Alg.2、Listing 1/2；PDF第6、9页

第12页 Verification：四项检查缺一不可

建议用时：75秒

验证端的工作可以拆成四项，首先验证 π 对应的电路约束确实成立，其次确认公开设备身份与签名恢复出的身份一致，而以太坊原型具体使用 pubAddr 等于 msg.sender 来检查提交者。

第三项确认公开根属于制造商认可的根集合，否则攻击者完全可以为自己随便编一个响应建树，再生成数学上有效但业务上无效的证明。

第四项检查挑战等于 latestChallenge，它把证明限制到当前周期，避免把旧周期的好状态当作当前状态；这并不自动定义同一周期内的重复提交是否允许，仍需应用层处理。

新鲜性还有一个运行前提，即未来挑战没有提前泄露，并且每个新挑战按时公布，论文因此提出由多方以阈值方式共同准备和发布挑战。

阈值分发分散的是挑战保密与发布可用性的信任，它和 Groth16 证明系统本身的可信初始化是两个问题。

内容出处：论文Alg.3、Listing 3、IV-B；PDF第7、10页

第13页 安全分析：保护机制与覆盖边界

建议用时：60秒

论文允许攻击者控制网络消息和普通软件层，但假设密码学工具以及 ROM、MPU 信任锚保持安全，在这个范围内，没有正确响应就不能为认可树生成有效成员证明。

对旧证明或者其他设备的证明，挑战绑定、身份绑定和最新挑战检查共同限制重放；对消息修改，数字签名、交易身份和 ZK 验证提供完整性检查。

DoS 方面的改进主要是制造商不用再处理逐设备在线请求，因此移除了一个集中式服务瓶颈，而论文明确没有覆盖所有链上合约 DoS 和底层基础设施攻击。

软件更新需要同步维护新根、软件与挑战，物理捕获和硬件篡改则超出论文范围。

把这些前提列出来并不是否定协议，而是使“ 安全 ”这个结论有清楚的范围，避免把周期性完整性证明误讲成持续安全保证。

内容出处：论文V-A/V-B；PDF第7 – 9页

第14页 实验设置与存储：规模增长体现在哪

建议用时：70秒

论文在一台 i5 笔记本、512 MB 内存的 Raspberry Pi Zero 2W，以及两 GB 内存的 Tinker Board 上测试原型，它们覆盖了通用处理器和两类小型 ARM 板，但并没有证明极低端微控制器也能运行。

实验主要测的是设备端 Attestation，Artifact Appendix 特别说明离线 Setup 没有计入整体性能评估，所以不能用这些数字判断制造商为大规模系统初始化需要多久。

表中树高从十增加到四十时，证明文件大小始终约八百零五字节，但 witness 从约八十七 KB 增加到约三百二十一 KB，证明密钥从一点六 MB 增加到六 MB，约束数接近一万。

这说明短证明有利于传输和验证，生成证明时的输入、密钥与电路成本却仍然增加。

这里的八百零五字节是论文原型记录的 proof 文件大小，不能把它直接视为所有 Groth16 编码的理论最小长度。

内容出处：论文Table III/IV、Artifact Appendix C；PDF第11、17页

第15页 原型链路：从电路到 Sepolia 合约

建议用时：55秒

原型的工具链可以按四个环节理解：Circom 编写并编译响应承诺与 Merkle 检查电路，snarkjs 和 Groth16 生成证明参数与验证器，设备脚本生成 witness 和证明，最后 Solidity 合约接收交易并完成四项检查。

论文在 Ethereum Sepolia 测试网部署了验证合约，并展示了添加根、发布挑战和设备提交证明的交易，因此它提供的是一套可运行的概念验证。

这里有两类容易混淆的准备，一类是 zRA 为认可响应建树并准备未来挑战，另一类是 Groth16 证明系统自身的可信初始化，公开透明并没有消除后一类要求。

区块链主要承担公开参数、证据发布和自动验证的基础设施角色，协议也可以考虑其他公开数据环境。

后面的本地验证耗时不能直接当作链上端到端确认时间，因为传播、出块、费用和最终确认属于另外一组成本。

内容出处：论文VI、Fig.9、Listing 2/3；PDF第9 – 10、17页

第16页 设备端耗时：论文结果与已有复现

建议用时：100秒

这一页把两组来源分开，左边是论文 Table V，右边是原 PPT 已经放入的 i7-12700KF 复现截图，我们这次只是读取并整理截图，没有重新运行实验，其中 WIT 表示见证生成，PRF 表示证明生成，单位都是秒。

以树高四十为例，Pi 的两个阶段是十一点三秒和十四点九秒，合计二十六点二秒；论文 i5 的两个阶段是零点零一秒和零点七六秒，合计约零点七七秒，设备差异非常明显。

已有复现每组运行三十次，在共同树高下 PRF 大约在零点五到零点六四秒之间，与论文笔记本处于相近量级，但 WIT 约零点二二到零点三秒，明显高于论文给出的零点零一秒。

不能因此马上断言协议或者复现有问题，因为处理器、witness 生成后端、工具版本、编译优化和计时口径都可能影响结果，而截图没有提供足够日志来排除这些因素；复现 PRF 在树高二十到三十之间还略有下降，所以也不能说结果严格单调。

论文正文声称笔记本 proof 不足四毫秒，但与 Table V 的秒级数据矛盾，这里采用表格作为汇报依据，并把不一致之处列入稿后的数据核对记录。

内容出处：论文Table V；原PPT第16页原始截图；PDF第11页

第17页 可扩展性：制造商轻了，全网仍有成本

建议用时：70秒

通信优势应当看清统计对象：如果有 n 台设备、每台证明 m 次，制造商只需发布 m 个共享挑战，因此这一项周期性发布量与设备数无关，而逐设备发挑战会带来 n 乘 m 级别的请求量。

举例来说，一百万台设备分别证明一万次，制造商仅发布一万个全局挑战，但所有设备仍然需要提交约一百亿份证明，链上吞吐、存储和费用不会因此自动消失。

离线 Setup 的响应与承诺规模也随 n 乘 m 增长，所以论文的可扩展性改进是移除集中式在线瓶颈，不是把系统的全部成本降为常数。

未来挑战的私有存储只随挑战数量增长，不过论文把一百万乘三十二字节写成约三 MB，正确计算应是三十二 MB，也就是约三十点五 MiB，公开树的存储要另外计算。

由于设备需要便捷地访问公开节点，论文认为 Pub/Sub 和 P2P 更适合，Mesh 或 Swarm 场景的表现还不能从这些分析直接推导出来。

内容出处：论文VII-B/C、Fig.6/7；PDF第11 – 12页

第18页 能耗与设备负担：快不一定更省电

建议用时：65秒

Table VI 从另一个角度提醒我们，耗时和能耗不能只选一个看，Pi 一次证明大约二十一点八秒、十四点四六焦耳，而 Tinker 大约十一点九秒，却消耗五十三点零八焦耳，按表中数值计算约为 Pi 的三点六七倍。

笔记本的零点四七九焦耳来自 powertop 估计，小板的数字来自 USB 电压电流测量，测量方法不同，因此跨平台比较还应带上这个限定。

论文给出的验证端小于一毫秒是计算性能指标，不能据此承诺 Sepolia 上的一次证明交易也能在一毫秒内被最终确认。

对电池设备，高频证明可能把计算与能耗压力直接转移到终端，而论文也指出极受限 AVR 或 Cortex-M 无法承担当前原型的证明计算。

因此这类方案更需要结合设备资源和证明频率判断是否适合，并继续研究更高效的证明后端、硬件加速或安全外包。

内容出处：论文Table VI、IX；PDF第12 – 14页

第19页 论文评价：贡献、代价与后续方向

建议用时：65秒

我认为论文最有价值的贡献，是把可信方掌握的正确响应转化为一份公众能够独立检查的零知识证据，同时用共享挑战降低制造商的重复在线通信，并提供了实际可运行的原型。

它改变的是验证权限和在线依赖的结构，信任并没有完全消失，而是集中在测量锚、认可响应的初始化、挑战发布，以及所选证明系统和公开基础设施的安全假设中。

代价方面，论文没有把 Setup 纳入性能实验，设备端的 witness、证明与能耗仍然明显，而且当前原型不能直接覆盖极低端微控制器、所有网络拓扑或大规模链上吞吐。

后续工作可以围绕轻量证明、硬件支持、安全外包和不同网络结构开展，但应分别检验正确性、隐私、新鲜性和实际部署成本。

最后也要保持对实验表述的核对，正文与表格不一致时应明确采用哪一组数据，而这篇论文给我们的总体启示是：真实状态从哪里来，和公众如何验证证据，是两层需要分别论证的问题。

内容出处：论文IV-B、V、IX；正文/表格一致性核对

第20页 谢谢聆听与问答

建议用时：10秒

我们的汇报到这里结束，zRA 的核心是用公开证明连接可信测量和独立验证，同时保留清楚的信任与性能边界。

谢谢大家，欢迎提问。

内容出处：原PPT第20页；论文结论

附录一：答辩问答

1. 公开透明是不是没有可信初始化？

不是。透明指公众可独立核验协议证据；Groth16 仍有证明系统的可信初始化要求，zRA 还需要准备正确响应承诺以及按时发布挑战。阈值挑战机制分散的是未来挑战的保密与发布可用性信任，不能自动替代证明系统初始化。

2. ZK 电路是否重新测量整台设备？

不直接重新测量。原型电路检查 response 的 Poseidon 承诺和 Merkle 成员路径；真实测量与响应生成由 ROM/MPU 信任锚保证。这个分工是判断安全结论的核心。

3. 为什么不能让 response 成为公开输入？

Listing 2 的公开列表只有 root、pubAddr、challenge；response 和路径作为见证。公开展示 response 并不是原型的验证接口，也会破坏“只提交公开证明、不展示见证”的讲解。

4. 为什么还要检查身份、根和最新挑战？

数学上有效的成员证明不等于业务上认可的设备证明。身份防止混用设备，validRoots 排除攻击者自建树，最新挑战防止用旧周期证据冒充当前证明；同周期内重复提交要另外定义策略。

5. 制造商是不是以后完全不需要在线？

不处理逐设备的在线证明请求，但仍需按时发布新挑战，软件更新也需要维护根与参数。论文阈值机制可分散挑战发布的可用性与保密信任，不能把未来挑战一次性提前公开。

6. 私有见证是不是必须放在 TEE 中？

不等同。私有见证表示证明接口不公开它；论文认为 response 的哈希形式不直接暴露内部状态，ZK 证明可在不可信软件层生成。真正受保护的是产生真实响应所需的测量程序和内部状态。

7. O(m) 的通信优势能否保证大规模链上部署？

不能直接保证。O(m) 指制造商的全局挑战发布量，设备总体提交、离线响应预计算和承诺树仍随 $n \times m$ 增长；链上吞吐、费用、节点访问和存储须另行评估。

8. 为什么原PPT复现的 witness 更慢？

目前只能提出候选原因：硬件、witness 后端、Node/snarkjs 版本、编译优化与计时口径。原截图只展示 CPU 型号、测试次数和均值，没有完整环境日志，因此不能断言某个原因，也不能把它视为本次重新复现。

9. 软件更新、rollback 与物理攻击怎么办？

论文建议更新二进制、认可根及后续挑战，防止旧软件响应被接受；具体部署还须维护根的认可与更新策略。设备捕获、硬件篡改不在论文覆盖范围，周期性测量也不保证两次证明之间的连续安全。

10. 能否用于最小型电池 IoT？

现有概念验证不支持泛化到所有低端设备。论文明确指出 AVR/Cortex-M 等极受限平台难以完成当前证明计算；需要结合频率、电源和资源预算，研究更高效后端、硬件加速或安全外包。

附录二：数据与表述核对

笔记本 proof 时间

正文 VI-C 有“ 不到4毫秒 ”的表述；Table V 的 i5 PRF 为0.51 – 0.76秒。主讲采用 Table V，h40 两阶段合计0.77秒。

未来挑战存储

论文 VII-C 写1,000,000×32 Bytes约3 MB；实际32,000,000 B，约32 MB（30.5 MiB）。公开树存储另计。

哈希输入顺序

式(1)/Alg.1写 H^3(pk,r_i,c_i)，Listing 2 实现顺序是(pk,c_i,r_i)。第10页注明协议记号，第11页注明实现顺序，实际Setup与电路必须一致。

proof 大小的含义

Table IV 报告约805B的原型proof文件大小，不是所有Groth16证明编码的理论最小值。

验证 <1 ms

Table VI 是验证计算指标，不能等同于Sepolia端到端确认时间；出块与最终确认不能沿用论文过时概述作为当前链的承诺。

复现结果来源

原PPT第16页截图：i7-12700KF，每项30次；本次只整理已有结果。共同高度h10/20/30/40的WIT为0.2194/0.2246/0.2632/0.2951 s，PRF为0.5023/0.5668/0.5480/0.6420 s。

能耗口径

Table VI的i5 479mJ换算为0.479J；powertop估计与USB测量的方式不同。Tinker/Pi能耗比53.08/14.46≈3.67，由表中数值计算。

通信图的基数

Fig.6/7的图注与正文存在设备数/证明次数基数不一致；第17页采用清楚定义的n=10^6、m=10^4示例，不照抄混淆的基数。